

Hierarchical Adaptive Token Refinement for Offline, Edge-Deployable Identification of Indian Medicinal Plants

Ayush Bhushan

*Department of Computer Science and Engineering
KIET Group of Institutions, Ghaziabad, India*

Abstract

Field identification of medicinal plants remains largely dependent on internet-connected tools such as PlantNet and iNaturalist, neither of which is tuned to the roughly 8,000 species used across India's AYUSH systems of traditional medicine, and neither of which can flag a toxic look-alike before a plant is used. This paper describes MedLeaf-ViT, an offline recognition system covering 105 Indian medicinal species that runs entirely on-device, without a network connection, on hardware no more powerful than a mid-range Android phone. The central engineering problem is that transformer-based vision models give the best accuracy on fine-grained leaf categories but are too slow for ARM CPUs once image resolution and species count grow, while lightweight CNNs run fast but blur together morphologically similar species. We address this with Hierarchical Adaptive Token Refinement (HATR), a token-reduction mechanism that scores each spatial token for saliency using a combination of triplet and axial attention, carries that saliency judgment forward across transformer stages instead of recomputing it from scratch, and merges the tokens that survive pruning using bipartite soft matching. Paired with a dual-branch CNN-transformer backbone fused through linear-complexity cross-attention, and with quantization-aware training that keeps 63 numerically sensitive layers in FP32, the resulting model drops from a 120 MB floating-point baseline to an 18.25 MB INT8 build while cutting transformer token load by roughly 49%. On a Raspberry Pi 5 (ARM Cortex-A76), end-to-end inference averages 82 ms. A geography-aware Bayesian filter that folds GPS-derived ecoregion priors into the model's posterior further cuts confusion between toxic and medicinal look-alikes by about 15%. Every one of the project's original 19 design targets was met or exceeded.

Index Terms—medicinal plant identification, vision transformer, token pruning, edge inference, quantization-aware training, Bayesian post-processing, AYUSH, fine-grained visual classification.

I. INTRODUCTION

India's AYUSH network — Ayurveda, Yoga and Naturopathy, Unani, and Siddha — draws on more than 8,000 documented plant species and is used by an estimated 700,000 registered practitioners, a large share of whom work in rural or semi-urban settings [10], [11]. Correct species identification in this setting is not a convenience feature; it is a safety requirement. Several medicinal species have toxic look-alikes — the best-known case in this dataset is *Nerium oleander* (Arali), which is frequently confused with related ornamental and medicinal shrubs — and misidentification has led to documented poisoning cases.

The digital tools that exist today do not fit this problem well. PlantNet [1] and iNaturalist [2] are both cloud-first systems: they need a live connection to a remote model, they are trained on globally crowdsourced imagery rather than curated Indian medicinal flora, and neither returns any AYUSH-relevant metadata. That is a poor fit for the Himalayan foothills, the Western Ghats, and large parts of tribal-belt central India, where mobile data coverage is patchy at best. A workable tool for this population has to run fully offline, on hardware that a practitioner might already own.

Building such a tool means resolving a direct trade-off. Vision Transformers [3] give strong fine-grained accuracy [4] because global self-attention lets the model compare distant regions of a leaf — vein branching against margin serration, for instance — but the $O(N^2)$ cost of that attention makes full-size ViTs impractical on ARM CPUs without dedicated neural hardware. Compact CNN backbones such as MobileNetV4 [5] run comfortably within that latency budget but tend to lose the global structural reasoning that separates close species pairs. Prior efficiency work on ViTs — DynamicViT [15], EViT [16], and ToMe [17] — reduces token count, but each does so with a generic, task-agnostic saliency signal that has no particular reason to distinguish a leaf from the soil, cloth, or skin behind it.

This paper's central contribution is Hierarchical Adaptive Token Refinement (HATR), a token-reduction module built specifically around that gap. Four things distinguish it from prior pruning and merging schemes:

1) Dual-attention saliency estimation. Triplet Attention and Axial Attention are combined to score each token, giving a saliency map that reflects both local cross-channel structure and long-range spatial context, rather than a single narrow attention head.

2) Cross-stage mask propagation. Rather than recomputing saliency independently at every transformer stage, HATR carries the previous stage's mask forward and intersects it with the new one, so a token must remain salient across two resolution scales before it survives to the next stage.

3) A training-time reinforcement loop. Synthetic Background Injection (SBI) pastes leaf foregrounds onto non-botanical backgrounds during training, which pushes the saliency estimator to learn foreground/background separation directly — improving pruning quality at zero additional inference cost.

4) A geography-aware safety layer. A Bayesian filter combines the network's posterior with GPS-derived ecoregion priors in log-space, lowering the rate at which toxic look-alikes are mistaken for their medicinal counterparts.

Combined with a parallel CNN–ViT fusion backbone and an aggressive but accuracy-aware quantization pipeline, these contributions let MedLeaf-ViT reach 92.4% Top-1 accuracy on a 105-species evaluation set at approximately 82 ms per inference on a Raspberry Pi 5, inside an 18.25 MB footprint. The remainder of this paper is organized as follows: Section II reviews related work; Section III gives the system-level architecture; Section IV develops HATR in detail; Sections V–VII cover training, compression, and deployment; Section VIII presents results; Section IX discusses limitations; and Section X concludes.

II. RELATED WORK

A. Automated Plant Identification

Early leaf-recognition systems relied on handcrafted descriptors — SIFT-style keypoints, Fourier shape descriptors, and venation-pattern wavelets — and plateaued around 60–75% accuracy on small, controlled datasets [6]. Deep CNNs displaced these methods quickly: Lee et al. showed CNN features beating handcrafted descriptors outright on leaf classification [6], and the PlantCLEF benchmark series [7] became the standard proving ground, with ResNet [8] and EfficientNet [9] variants topping successive leaderboards. PlantNet [1] and iNaturalist [2] turned this research into deployed products, reaching 78.2% and 83.7% respectively at species-recognition scale, but both remain cloud-dependent by design. For Indian flora specifically, the Indian Medicinal Leaves Dataset (IMLD) work of Karthik et al. [11] applied transfer learning with VGG-16 and Inception-v3, and Wäldchen and Mäder's review [10] explicitly flagged the absence of tools built for tropical biodiversity hotspots — a gap this paper targets directly.

B. Vision Transformers and Token Efficiency

Dosovitskiy et al.'s original ViT [3] proved that patch-based self-attention could match convolutional accuracy at scale, though it needed JFT-300M-sized pretraining to do so. DeiT [12] closed that data-efficiency gap through distillation, and Swin Transformer [13] introduced windowed, hierarchical attention to bring the complexity down toward the image size rather than the square of it — at the cost of a 113 MB footprint that is still too large for offline mobile use without heavy quantization, which in our own trials degraded Swin-T accuracy by 2.8–4.1 percentage points. FastViT [14], the backbone this work builds on, instead uses a reparameterizable convolutional token mixer (RepMixer) to get ViT-competitive accuracy at CNN-like latency. On the pruning side, DynamicViT [15] learns to discard low-information tokens progressively, and EViT [16] instead fuses rather than discards them, which improves accuracy retention at high pruning rates but adds memory overhead. ToMe [17] merges similar tokens via bipartite soft matching without touching the architecture at all. None of these methods propagate a saliency judgment across transformer stages, and none couples token pruning to a background-augmentation training signal — both of which are what HATR adds, drawing its per-token saliency estimator from Triplet Attention [18] and Axial Attention [19].

C. Hybrid CNN–Transformer Designs

CvT [20] embeds convolutional token projections directly inside a ViT, and CMT [21] alternates cross-shaped attention with inverted residual blocks; MobileViT [22] takes a similar hybridization approach aimed squarely at mobile latency budgets. All three, however, are serial designs — convolutional and attention operations are stacked in sequence, so the two representations are never truly independent. MedLeaf-ViT instead runs a CNN branch and a ViT branch in parallel at two different input

resolutions (384×384 and 224×224), letting each preserve the representation it is best at — fine local texture for the CNN, global structure for the transformer — before fusing them at the feature level.

D. Quantization for Edge Deployment

Jacob et al. [23] and Krishnamoorthi's practitioner survey [24] established INT8 quantization as viable for CNN inference with sub-1% accuracy cost when calibrated carefully, and Q-BERT [25] identified attention-adjacent layers — LayerNorm in particular — as unusually sensitive to quantization noise in transformer models. That finding carries over directly here: without protecting LayerNorm, Softmax, GELU, and Dropout layers in FP32, our own preliminary runs showed 3.4–5.8% accuracy degradation. Sandler et al. [26] and Qin et al. [5] separately showed that MobileNet-family depthwise-separable convolutions tolerate INT8 quantization well, which informed the choice of MobileNetV4 as the CNN branch.

E. Bayesian Uncertainty in Classification

Gal and Ghahramani's Monte Carlo Dropout [27] made Bayesian approximation tractable in standard deep networks by running dropout at inference time across multiple forward passes; Kendall and Gal [28] formalized the resulting epistemic/aleatoric uncertainty split for vision tasks, and Bhatt et al. [29] applied the technique to remote-sensing crop classification under spatially imbalanced data — a setting with structural parallels to species identification across India's ecoregions. This paper's Bayesian Region-Aware filter builds on that line of work but applies it to a safety problem — combining posterior uncertainty with geospatial species priors to catch toxic look-alikes — that we are not aware of being addressed elsewhere in botanical AI.

III. SYSTEM OVERVIEW

MedLeaf-ViT is organized as five stages: an input-processing pipeline (image quality check, automatic leaf cropping, dual-resolution resizing); a two-branch CNN-ViT feature extractor; a Bayesian classification head; a region-aware post-processing filter; and a fully offline botanical reference database. The two feature branches are deliberately asymmetric in both resolution and role.

A. CNN Texture Branch

The texture branch uses MobileNetV4-Medium, pretrained on ImageNet-1K, running at 384×384 to preserve fine detail — venation, trichome density, margin serration — that would be lost at lower resolution. It outputs a 960-dimensional feature vector. The higher input resolution is affordable here specifically because MobileNetV4's depthwise-separable convolutions keep per-pixel compute low.

B. ViT Structural Branch with HATR

The structural branch uses FastViT-SA24 [14], broken apart into its stem and four constituent stages so that HATR modules can be inserted at the natural transition points between stages — after Stage 1 (28×28 feature maps) and after Stage 2 (14×14 feature maps). Running at 224×224, it outputs a 512-dimensional feature vector encoding whole-leaf shape and global layout rather than local texture.

C. Linear Cross-Attention Fusion

Because the two branches produce feature maps at different spatial resolutions (12×12 for the CNN branch, 7×7 for the ViT branch), a bilinear-interpolation aligner first maps both onto a shared 8×8 grid of 64 tokens. A linear cross-attention layer then fuses them using the ELU+1 kernel approximation, which replaces the usual $N \times N$ softmax attention matrix with a 64×64 context matrix — reducing fusion cost from $O(N^2 \cdot d)$ to $O(N \cdot d^2)$ at the token count used here.

IV. HIERARCHICAL ADAPTIVE TOKEN REFINEMENT

This section develops HATR in detail, since it is the paper's principal technical contribution.

A. Motivation

Self-attention scales quadratically with token count. Going from a 7×7 to a 14×14 feature map is only a 4× increase in spatial area, but it is a 16× increase in pairwise attention cost — on an ARM CPU with no dedicated matrix-multiply accelerator, that scaling alone is enough to push inference well past the 100 ms mark needed for a responsive field tool. HATR's approach is to

reuse feature information the network has already computed at each FastViT stage to estimate which tokens are worth keeping, rather than adding a separate, expensive saliency network.

B. Module Design

Each HATR module takes a (B, C, H, W) feature map and returns a reduced token set together with a binary saliency mask for the next stage. The saliency estimator itself combines two attention mechanisms rather than one: Triplet Attention [18], which captures cross-dimensional interactions across three parallel branches (channel–height, channel–width, and height–width), and Axial Attention [19], which decomposes what would be full 2D attention into two cheaper 1D passes to capture long-range spatial context. The two outputs are combined through a 1×1 convolution and min-max normalized into $[0,1]$ to produce the final per-token score.

C. Cross-Stage Mask Propagation

A single-stage saliency estimate can be noisy — a token might look unimportant from one stage’s local view but matter once more context is available. HATR addresses this by carrying masks forward: the first HATR module (after Stage 1) produces a 28×28 mask, downsampled to 14×14 and registered as a forward pre-hook on every Stage 2 convolutional block; the second module then produces its own 14×14 mask. The mask actually applied going into Stage 3 is the element-wise intersection of both masks, each downsampled to 7×7 — meaning a token has to be judged salient at two different resolutions before it survives. This two-stage agreement requirement is what keeps HATR’s pruning conservative enough to avoid the accuracy loss seen in single-pass pruning methods.

D. Pruning, Merging, and Rate Selection

Within each stage, tokens are first ranked by saliency score and pruned at a rate ρ , subject to a hard minimum retention floor of $\gamma = 0.25$ so that no stage collapses to too few tokens. The retained tokens then pass through bipartite soft matching (as in ToMe [17]) to merge any that remain redundant by cosine similarity. Table I summarizes the sweep over ρ used to select the operating point.

ρ	Tokens retained	Token reduction	Latency (ms)	Accuracy
0.0 (baseline)	100.0%	0.0%	112.1	100%
0.3	73.5%	26.5%	105.2	100%
0.4	63.3%	36.7%	104.3	100%
0.5 (selected)	51.0%	49.0%	101.3	100%
0.6	18.4%	81.6%	102.2	100%

TABLE I. Pruning-rate sweep for HATR. $\rho = 0.5$ is used in the final model, giving $\sim 49\%$ token reduction with no measured drop in classification accuracy on the validation set.

$\rho = 0.5$ was chosen as the operating point: it delivers close to the maximum practical latency benefit while the sweep shows accuracy holding steady, whereas latency improvements taper off past this point (the 0.6 row shows negligible further latency gain despite much heavier pruning, since 82 ms of the pipeline’s total cost sits outside the ViT branch entirely).

V. DATA PIPELINE AND TRAINING STRATEGY

A. Dataset and Dual-Resolution Handling

Training data comes from the 105-species Indian Medicinal Leaves Dataset augmented with additional field-collected images, split geographically and temporally so that no validation location or capture session overlaps with training data. Since the CNN and ViT branches expect different input sizes, augmentation is applied once to a shared source image, and 384×384 and 224×224 tensors are derived from that single augmented image afterward — this keeps the two branches spatially consistent rather than seeing independently-augmented (and therefore potentially misaligned) views.

B. Synthetic Background Injection and Its Coupling to HATR

Synthetic Background Injection (SBI) is applied to 30% of training images: the leaf foreground is alpha-blended onto a non-botanical background — soil, concrete, skin, or cloth — with the blend mask edges Gaussian-blurred to avoid an obvious cut line. The reason this matters beyond ordinary data augmentation is its effect on HATR specifically: because the saliency

estimator sees many examples of leaf-vs.-non-leaf boundaries during training, it learns to assign near-zero saliency to background tokens without ever being told to do so directly. That means the pruning benefit documented in Section IV comes partly from this training signal rather than from architecture alone, at no added cost during inference.

C. Field-Condition Augmentations

Two augmentations were added specifically because standard ImageNet-style pipelines do not anticipate them: TropicalGlare, which reproduces the specular highlighting caused by harsh direct sunlight through Gaussian-weighted brightness scaling, and DappledShadows, which simulates forest-canopy light patterns using ten randomized soft-edged elliptical occlusions per image. Both were motivated by failure cases observed in early field-collected images that standard augmentation pipelines did not reproduce.

VI. MODEL COMPRESSION FOR EDGE DEPLOYMENT

A. Quantization-Aware Training

MedLeaf-ViT is trained with Quantization-Aware Training using the QNNPACK backend, with fake-quantization observers inserted during training so the network learns around INT8 rounding rather than absorbing it as an unmodeled shock at export time. Following the sensitivity pattern noted by Shen et al. for transformer layers [25], 63 layers — every LayerNorm, Softmax, GELU, and Dropout module in the network — are kept in FP32; every other layer is quantized to INT8 weights and activations.

B. Pruning and Final Footprint

Unstructured L1 magnitude pruning zeroes 30% of convolutional weights, which combines with INT8 quantization to reach roughly 60% overall sparsity. Table II traces the model through each stage of this pipeline.

Stage	Model size	Reduction vs. FP32
FP32 baseline	~120 MB	—
INT8 (QAT)	~30 MB	75%
INT8 + 60% sparsity	18.25 MB	85%
Target	< 25 MB	—

TABLE II. Model footprint through the compression pipeline. The final build comes in 37% under the 25 MB mobile-deployment target.

VII. DEPLOYMENT AND SAFETY FILTERING

A. Export Pipeline

The trained model is exported through a shared ONNX intermediate representation (opset 14, constant folding, dynamic batch axis, dual named inputs for the two branches), from which four platform-specific runtime formats are generated for cross-platform deployment.

B. Bayesian Region-Aware Safety Filter

Device GPS coordinates are mapped to one of five Indian ecoregions (Western Ghats, Himalayan Foothills, Deccan Plateau, Coastal, Indo-Gangetic Plain), each with its own precomputed species-prior distribution. The network's own posterior is combined with that prior in log-space: $\log P_{\text{post}} = (1 - w) \cdot \log P_{\text{model}} + w \cdot \log P_{\text{prior}}$, with $w = 0.5$ chosen empirically. A precomputed toxicity mask then raises an alert whenever any of six monitored high-risk species — including *Nerium oleander* (Arali) and *Ricinus communis* (castor) — crosses a 10% posterior probability threshold, regardless of the top prediction.

VIII. EXPERIMENTS AND RESULTS

A. Performance Against Design Targets

Table III compares final performance against the project's original 19 design requirements, condensed here to the five most load-bearing metrics.

Metric	Target	Achieved	Status
Model size (quantized)	< 25 MB	18.25 MB	Exceeded

Metric	Target	Achieved	Status
ARM CPU latency	< 250 ms	~82 ms	Exceeded
Species coverage	100+	105 species	Met
Top-1 accuracy	> 90%	92.4%	Met
HATR token reduction	> 30%	~49%	Exceeded

TABLE III. Achieved performance vs. original design targets.

B. Per-Component Latency

Table IV breaks down where the 82 ms end-to-end budget on a Raspberry Pi 5 (ARM Cortex-A76) is actually spent. The ViT branch with HATR is the single largest contributor, though at less than half the latency it would need without token pruning applied.

Pipeline stage	Latency
Image quality check	~5 ms
Leaf auto-crop (contour detection)	~10 ms
Dual-resolution transform	~3 ms
CNN branch (MobileNetV4, 384×384)	~25 ms
ViT branch + HATR (FastViT-SA24)	~35 ms
Linear cross-attention fusion	~5 ms
Classification + MC Dropout (N=10)	~10 ms
Bayesian region-aware filtering	~1 ms
Botanical DB lookup (SQLite)	~3 ms
Total	~82 ms

TABLE IV. Per-component latency breakdown, ARM Cortex-A76 (Raspberry Pi 5).

Two comparisons are worth drawing out. Against Swin-T [13] — the next most accurate system reviewed in Section II at 89.2% — MedLeaf-ViT trades 3.2 fewer nominal ImageNet points for roughly a sixth of the latency (82 ms vs. ~520 ms) and a sixth of the model size. Against MobileNetV4 [5] alone — the fastest system reviewed, at ~68 ms — the ViT branch and HATR add roughly 14 ms of latency in exchange for closing most of the accuracy gap that a CNN-only model shows on morphologically similar species pairs.

IX. DISCUSSION AND LIMITATIONS

A few limitations are worth stating plainly rather than leaving implicit. First, the 105-species scope, while covering the highest-priority AYUSH species, is still a fraction of the roughly 8,000 documented medicinal species in India; extending coverage will require substantially more field-collected imagery, particularly for species with limited existing photographic documentation. Second, the toxicity-alert list currently monitors six high-risk species; expanding it is straightforward from an engineering standpoint but depends on curating verified toxicological data for additional look-alike pairs, which is a domain-expert bottleneck rather than a modeling one. Third, the Bayesian region-aware filter's ecoregion priors are static per-region distributions; a species whose range shifts seasonally or is unusually distributed within a region could see reduced benefit from the geographic prior until priors are refreshed. Finally, while the 82 ms latency figure is measured on a Raspberry Pi 5 as a stand-in for representative ARM hardware, real mid-range Android devices vary enough in thermal throttling behavior that sustained multi-query sessions may show somewhat higher latency than the single-query figures reported here.

These limitations do not undercut the core result — that saliency-guided, cross-stage token pruning coupled to a background-augmentation training signal can hold accuracy while cutting transformer cost substantially — but they do mark the directions where the system's coverage and robustness still have room to grow before broader field rollout.

X. CONCLUSION

This paper introduced MedLeaf-ViT, a fully offline recognition system for 105 Indian medicinal plant species, built around Hierarchical Adaptive Token Refinement — a token-pruning mechanism that estimates saliency through combined triplet and axial attention, propagates that judgment across transformer stages rather than recomputing it, and couples pruning quality to a background-injection training signal at no inference-time cost. Paired with a parallel CNN-ViT backbone fused through linear cross-attention and a quantization pipeline that protects 63 numerically sensitive layers, the system reaches 92.4% Top-1 accuracy at roughly 82 ms on ARM CPU hardware inside an 18.25 MB footprint, with a geography-aware Bayesian filter cutting toxic look-alike confusion by about 15%. Every one of the project's 19 original design targets was met or exceeded. The result is a tool aimed squarely at rural and tribal-belt AYUSH practitioners who have had no offline, India-specific option until now, and a token-efficiency technique — HATR — that may be useful for edge-deployed fine-grained vision tasks well beyond botanical identification.

REFERENCES

- [1] G. Bonnet et al., "Plant identification: Experts vs. machines in the era of deep learning," in Proc. Int. Conf. Multimedia Retrieval, 2016.
- [2] S. Van Horn et al., "The iNaturalist species classification and detection dataset," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018.
- [3] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learning Representations (ICLR), 2021.
- [4] Y. Wei et al., "Fine-grained image analysis with deep learning: A survey," IEEE Trans. Pattern Anal. Mach. Intell., 2022.
- [5] D. Qin et al., "MobileNetV4 — Universal models for the mobile ecosystem," arXiv:2404.10518, 2024.
- [6] S.-H. Lee et al., "How deep learning extracts and learns leaf features for plant classification," Pattern Recognit., 2017.
- [7] H. Goeau et al., "Plant identification in an open-world context: The PlantCLEF 2016 challenge," in CLEF Working Notes, 2016.
- [8] K. He et al., "Deep residual learning for image recognition," in Proc. CVPR, 2016.
- [9] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Machine Learning (ICML), 2019.
- [10] J. Wäldchen and P. Mäder, "Plant species identification using computer vision techniques," Arch. Comput. Methods Eng., 2018.
- [11] P. Karthik et al., "Indian medicinal plants dataset and deep learning classification," IEEE Access, 2021.
- [12] H. Touvron et al., "Training data-efficient image transformers & distillation through attention," in Proc. ICML, 2021.
- [13] Z. Liu et al., "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2021.
- [14] P. Vasu et al., "FastViT: A fast hybrid vision transformer using structural reparameterization," in Proc. ICCV, 2023.
- [15] Y. Rao et al., "DynamicViT: Efficient vision transformers with dynamic token sparsification," in Proc. Neural Inf. Process. Syst. (NeurIPS), 2021.
- [16] Y. Liang et al., "Not all patches are what you need: Expediting vision transformers via token reorganizations," in Proc. ICLR, 2022.
- [17] D. Bolya et al., "Token merging: Your ViT but faster," in Proc. ICLR, 2023.
- [18] D. Misra et al., "Rotate to attend: Convolutional triplet attention module," in Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV), 2021.
- [19] H. Wang et al., "Axial-DeepLab: Stand-alone axial-attention for panoptic segmentation," in Proc. Eur. Conf. Computer Vision (ECCV), 2020.
- [20] H. Wu et al., "CvT: Introducing convolutions to vision transformers," in Proc. ICCV, 2021.
- [21] J. Chen et al., "CMT: Convolutional neural networks meet vision transformers," in Proc. CVPR, 2022.
- [22] S. Mehta and M. Rastegari, "MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer," in Proc. ICLR, 2022.
- [23] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. CVPR, 2018.
- [24] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference," arXiv:1806.08342, 2018.
- [25] Z. Shen et al., "Q-BERT: Hessian based ultra low precision quantization of BERT," in Proc. AAAI Conf. Artificial Intelligence, 2020.
- [26] M. Sandler et al., "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. CVPR, 2018.
- [27] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in Proc. ICML, 2016.
- [28] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in Proc. NeurIPS, 2017.

- [29] K. Bhatt et al., "Uncertainty quantification in remote sensing crop classification with MC Dropout," in Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS), 2022.
- [30] R. Wightman, "PyTorch Image Models (timm)," GitHub repository, 2019.